

Beauty And The Beast

Exploiting GPUs In Haskell

Alex Cole

University of Leicester

CPA 2012, Dundee

27th August 2012

Alistair A McEwan

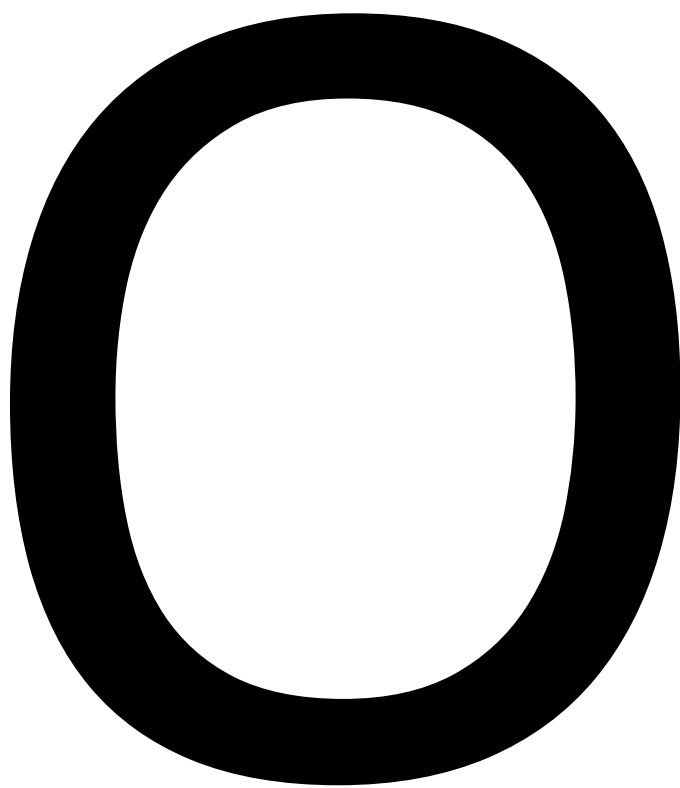
University of Leicester

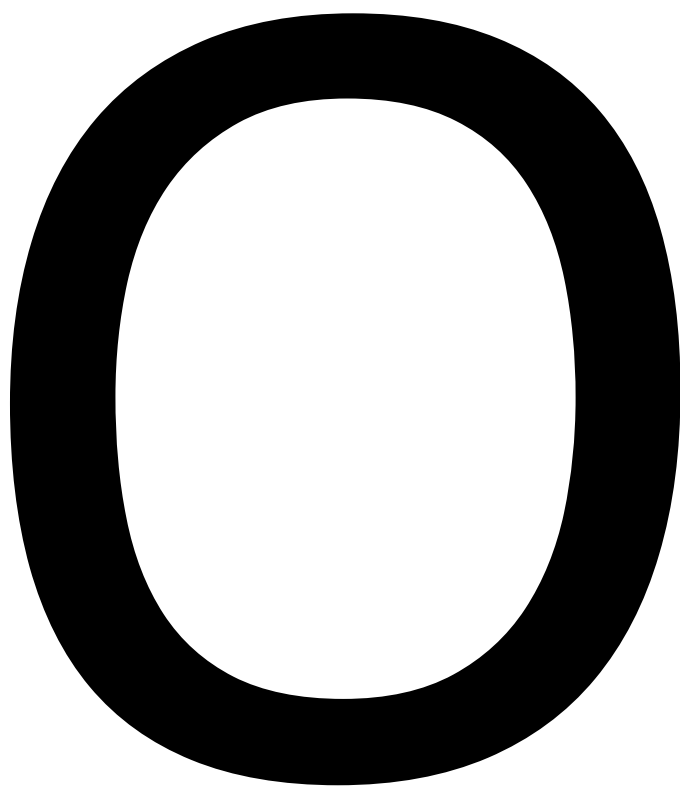
Geoff Mainland

Microsoft Research



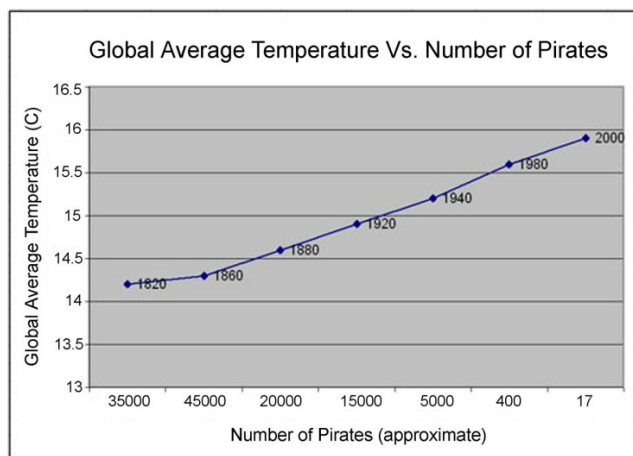
Z

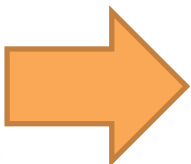
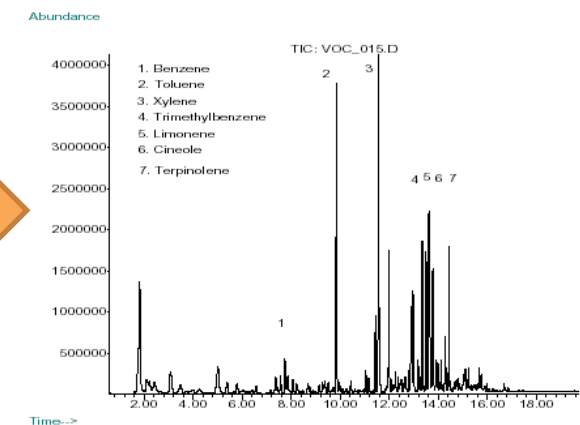


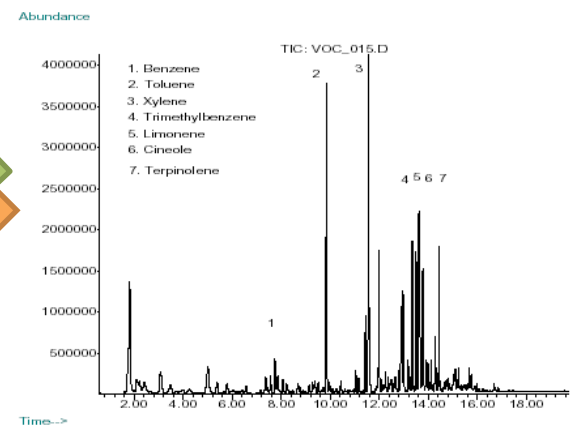


M

!









$$(x + a)^n = \sum_{k=0}^n \binom{n}{k} x^k a^{n-k}$$

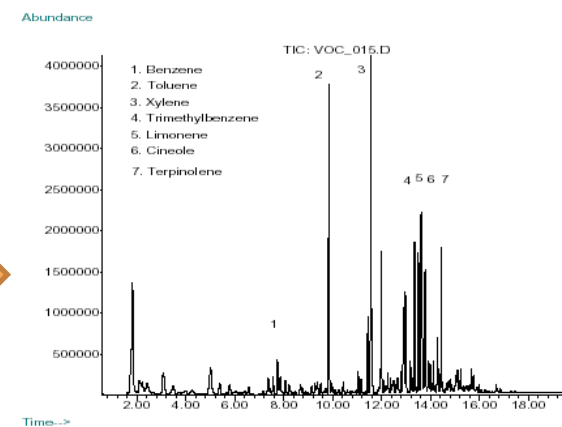
[illegible]

$$(x + a)^n = \sum_{k=0}^n \binom{n}{k} x^k a^{n-k}$$

[illegible]

$$(x+a)^n = \sum_{k=0}^n \binom{n}{k} x^k a^{n-k}$$

1st Weight Reading	2nd Weight Reading	3rd Weight Reading	Average Weight	1st Weight Reading	2nd Weight Reading	3rd Weight Reading	Average Weight	Body Mass Index
1.345	1.345	1.310	1.333	40.0	40.0	39.0	39.7	22.72
1.660	1.670	1.630	1.653	50.0	50.0	49.0	49.7	28.33
1.650	1.650	1.630	1.643	50.0	50.0	49.0	49.7	28.33
1.700	1.720	1.710	1.710	50.0	52.0	51.0	51.0	29.03
1.650	1.650	1.630	1.643	50.0	50.0	49.0	49.7	28.33
1.750	1.770	1.770	1.770	55.0	57.0	57.0	57.0	30.08
1.800	1.800	1.800	1.800	60.0	60.0	60.0	60.0	31.11
1.850	1.850	1.870	1.870	65.0	65.0	67.0	65.7	32.08
1.875	1.875	1.890	1.890	66.0	66.0	67.0	66.3	32.57
1.900	1.900	1.900	1.900	66.0	66.0	66.0	66.0	32.57
1.950	1.950	1.965	1.965	68.0	68.0	69.0	68.3	33.58
1.975	1.975	1.975	1.975	68.0	68.0	68.0	68.0	33.58







[Only] a few of the proposed approaches [...] are currently used in practice due to the unavailability of the software or due to difficulties with incorporating the new tools in the existing data analysis pipelines.

- Nesvizhskii10



Hypotheses

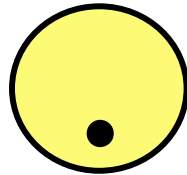
- Haskell is easy
- But performance (lists) stinks
- Using a DSL we can recover some “C” performance
- We think the trade-off is acceptable
- Based on this, we can address the richer MS computation domain with ease and performance.





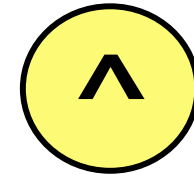
Whole Array

/ Index



- Shift $.<<<$
- Add $.+$
- Multiply $.^*$

Per-Element



- Shift $^<<<$
- Add $^+$
- Multiply *



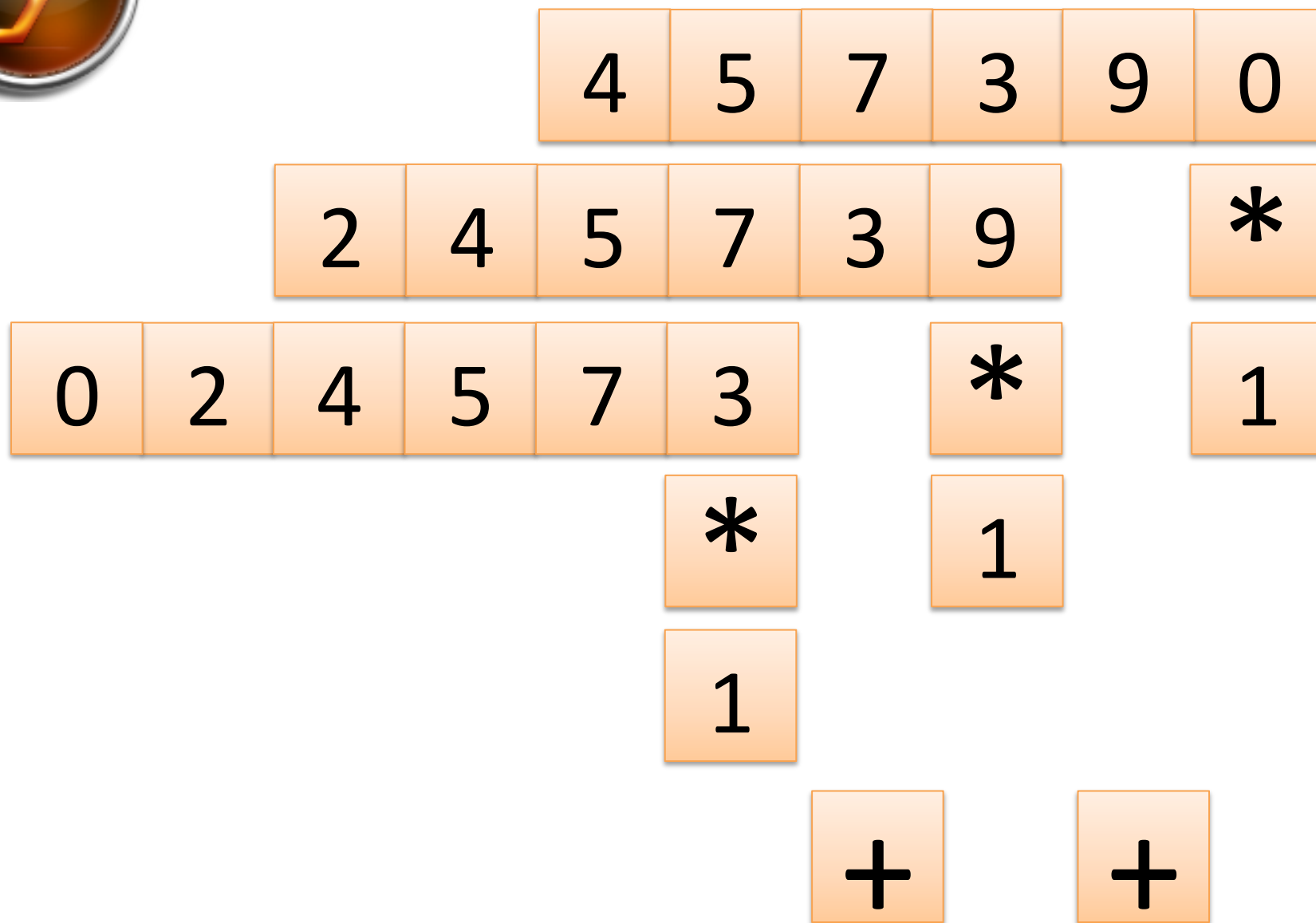
2 4 5 7 3 9

*

1 3 1

=

10 19 18 29 25 30





```
-- Sum up all the parts for the final answer.
```

```
convolve filter input = sum convolvedParts
```

```
where
```

```
-- Convolve the whole array with one filter element and shift and multiply.
```

```
oneFilterElement shift mul = (input .<<< shift) ^*
```

```
-- Get the filter radius.
```

```
r = length filter `div` 2
```

```
-- Do all the shifts and multiplies separately.
```

```
convolvedParts = zipWith oneFilterElement [-r..r] filter
```



- Using Lists:

```
convolve :: [Float] -> [Float] -> [Float]
```

- Using PEGGY:

```
convolve :: [Float] -> PYExpr Float target -> PYExpr Float target
```

```
-- Convert the old input to PEGGY GPU data.
```

```
peggyInput = toArray listInput
```



```
data PYExpr a b = (PYCode a b) (PYMem a b)
```

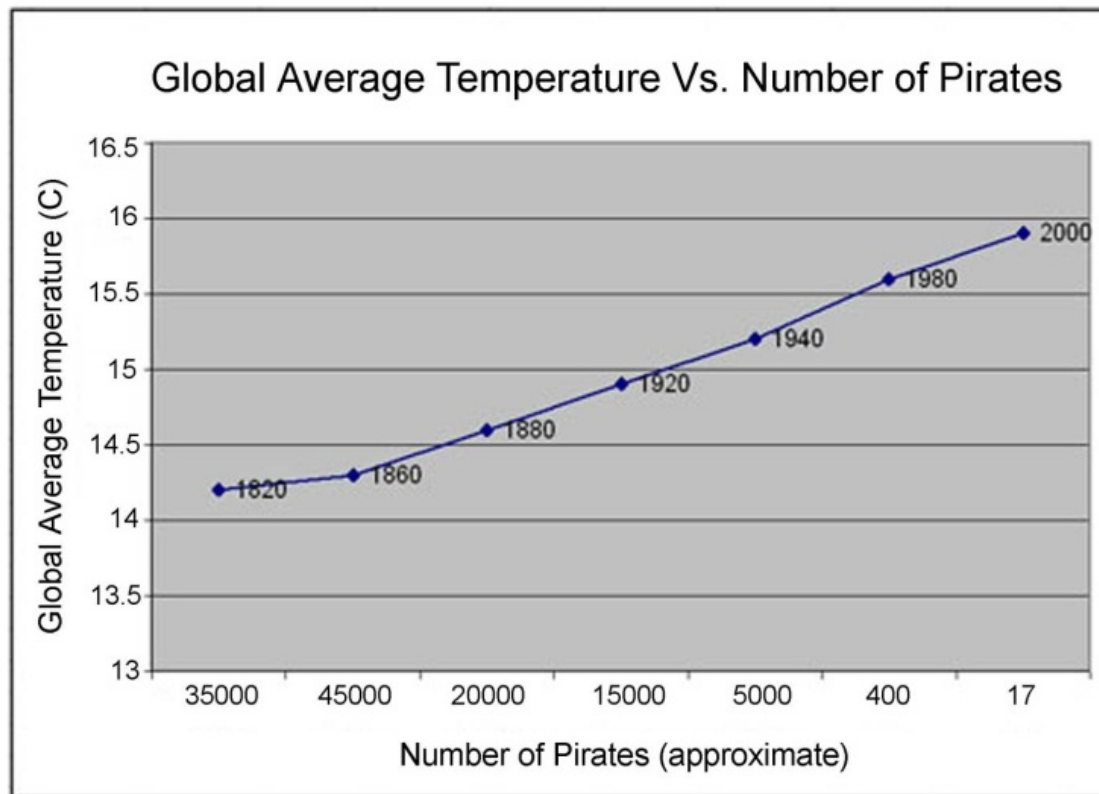
```
data instance PYMem a TargetHaskell = [a]
```

```
instance PYCode a TargetHaskell where
```

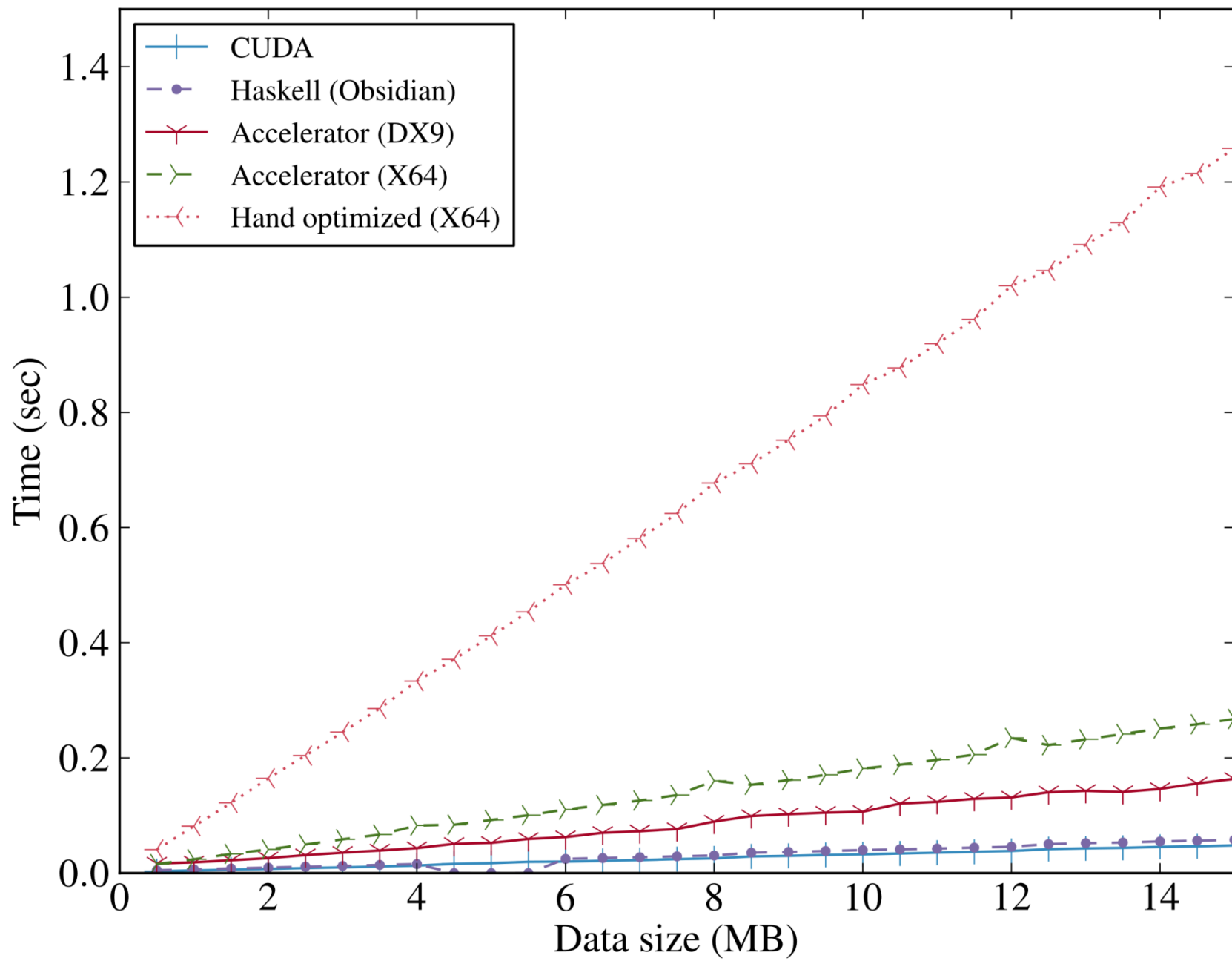
```
    (^+) x y = zipWith (+) x y
```

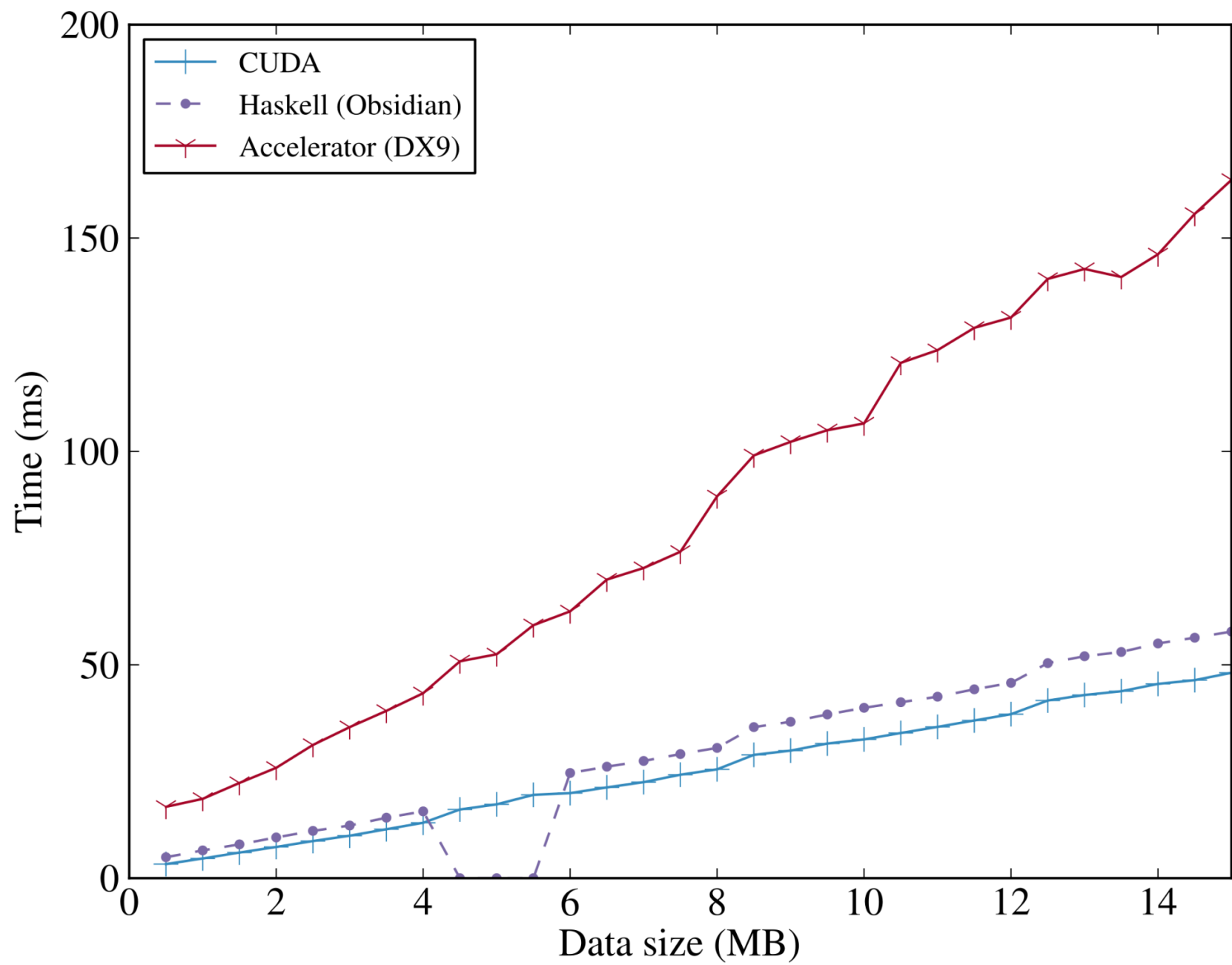


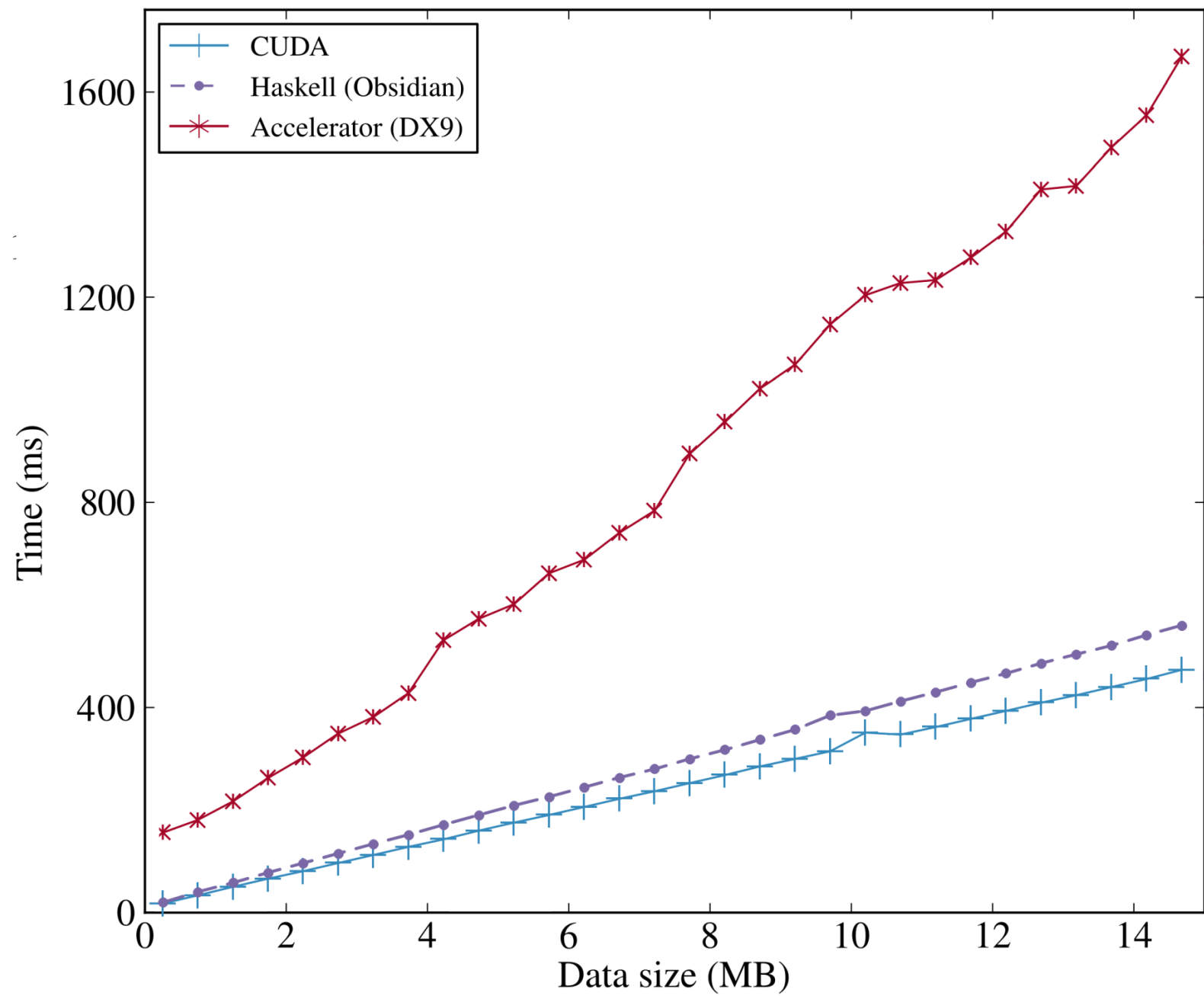
STOP GLOBAL WARMING: BECOME A PIRATE

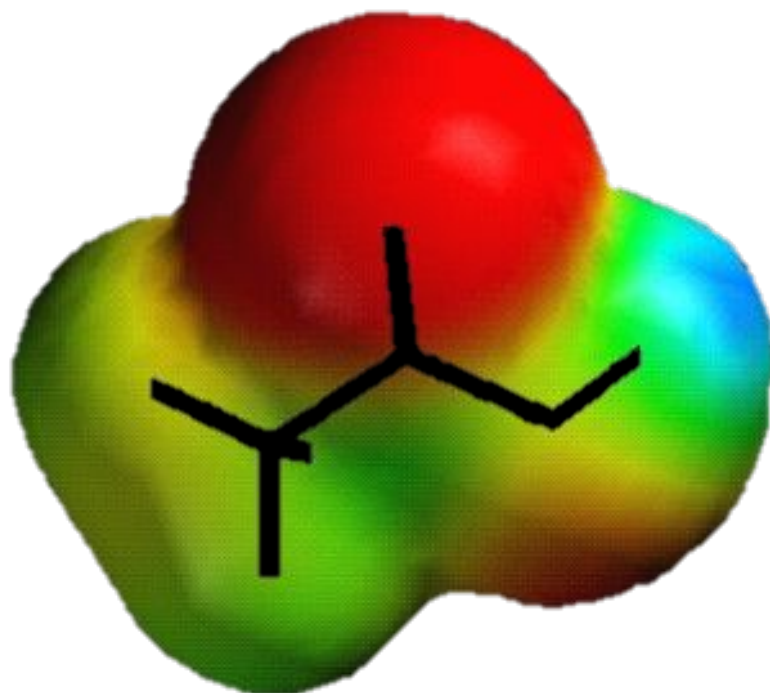


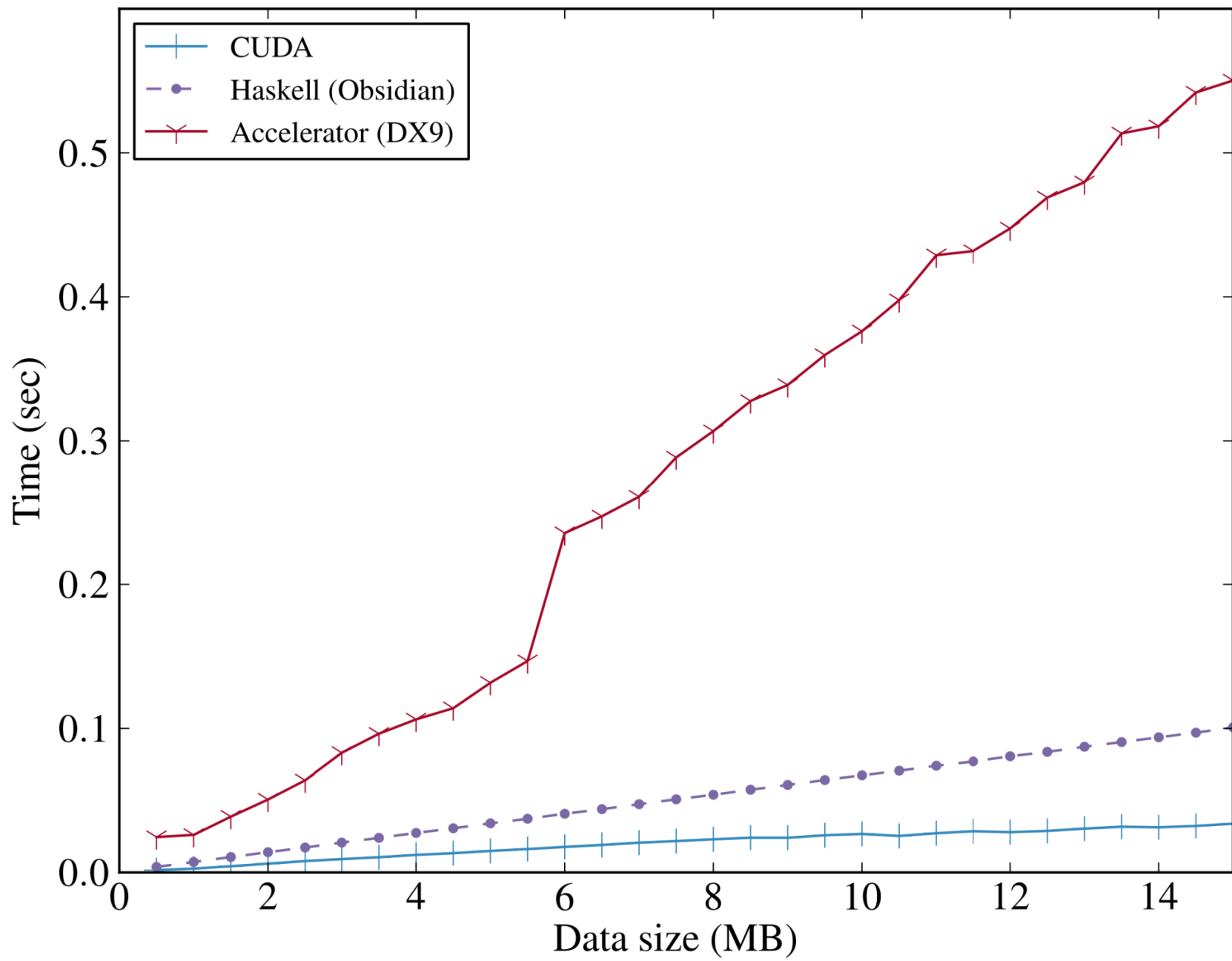
WWW.VENGANZA.ORG













DATA

* f1 =

* f2 =

LOW FREQUENCY

HIGH FREQUENCY

* f1 =

* f2 =

LOW FREQUENCY

HIGH FREQUENCY

* f1 =

* f2 =

LOW FREQUENCY

HIGH FREQUENCY

f1

f2

LOW
FREQUENCY

HIGH
FREQUENCY



CUT OFF FILTER

DATA



DATA

* f3 =

* f4 =

LOW FREQUENCY

HIGH FREQUENCY

* f3 =

* f4 =

LOW FREQUENCY

HIGH FREQUENCY

* f3 =

* f4 =

LOW FREQUENCY

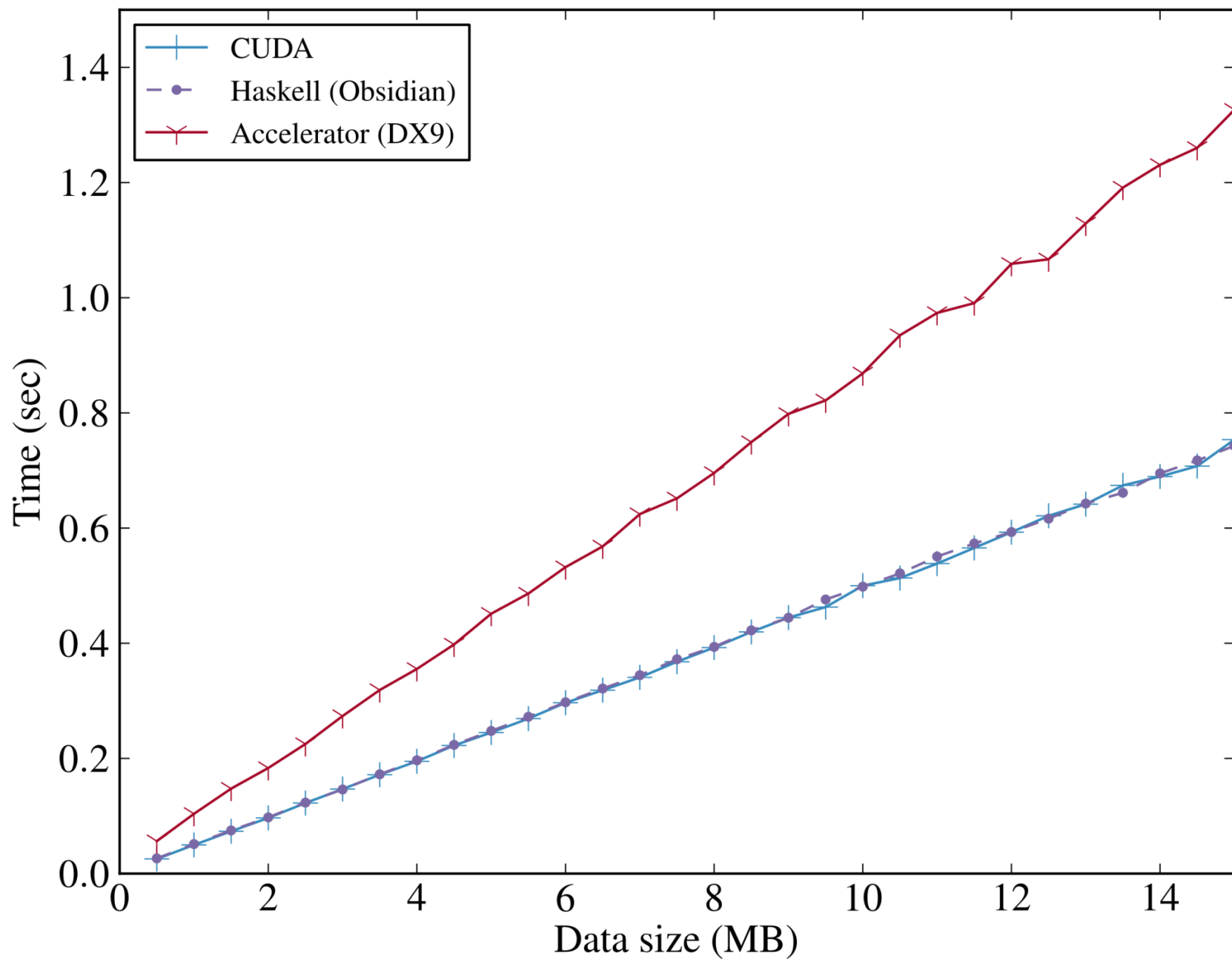
HIGH FREQUENCY

f3

f4

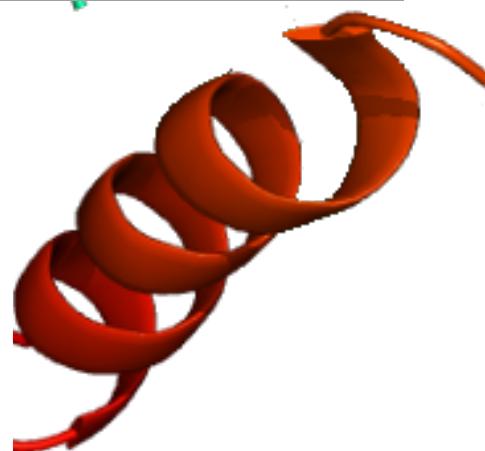
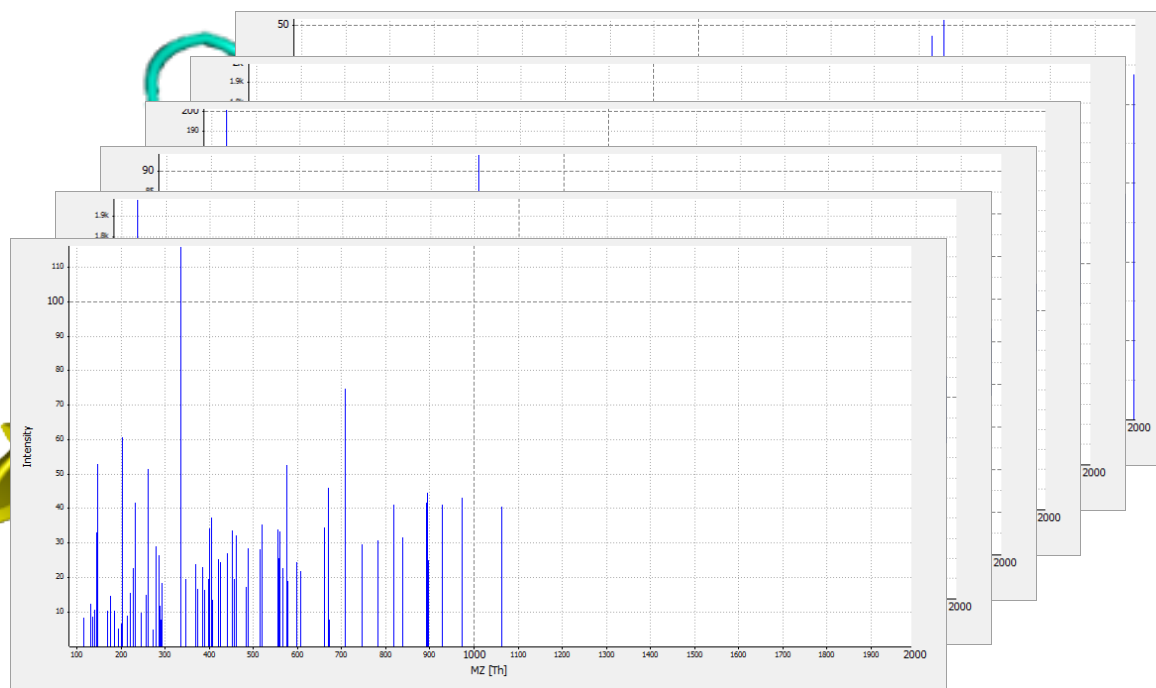
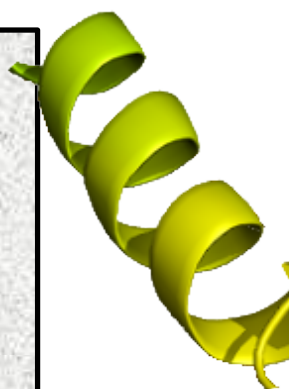
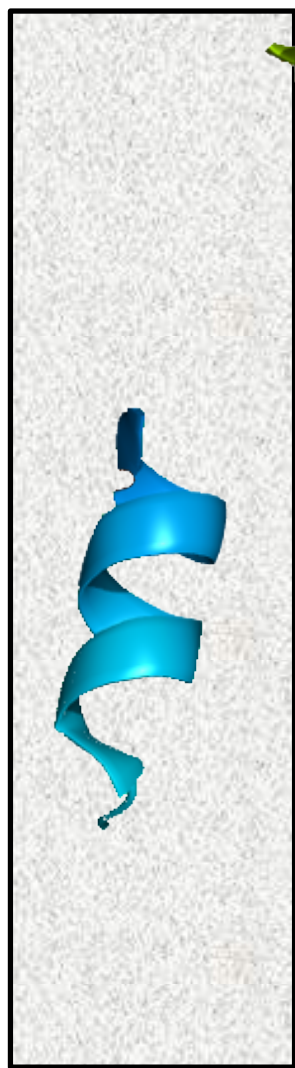
LOW
FREQUENCY

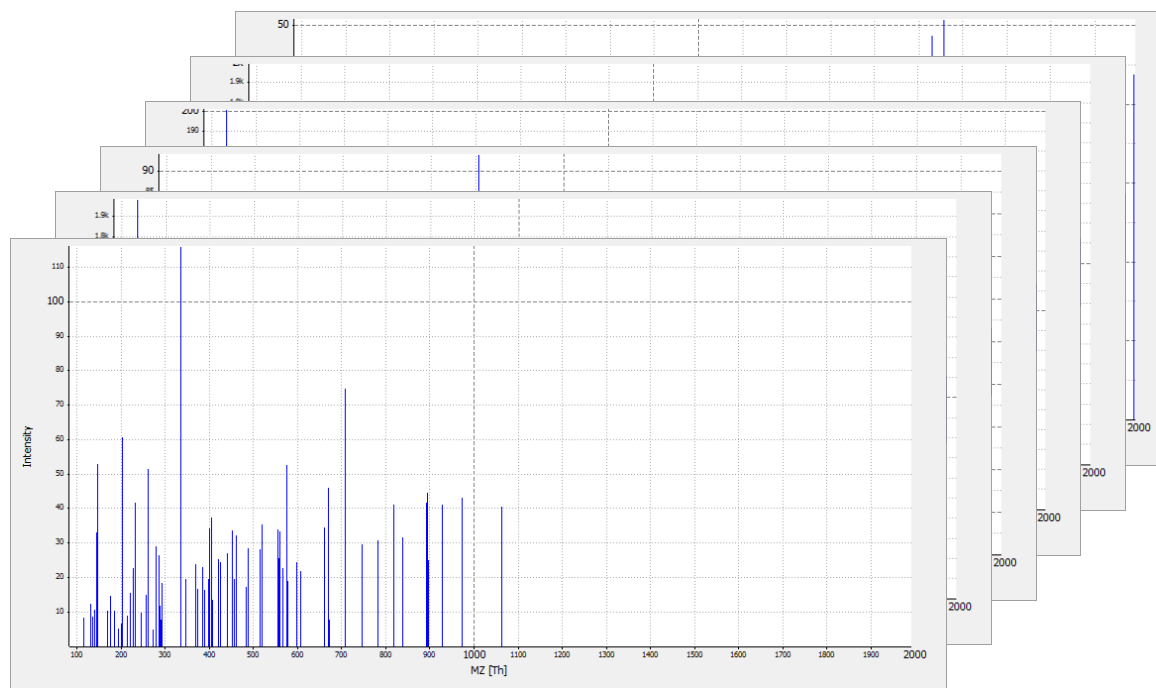
HIGH
FREQUENCY





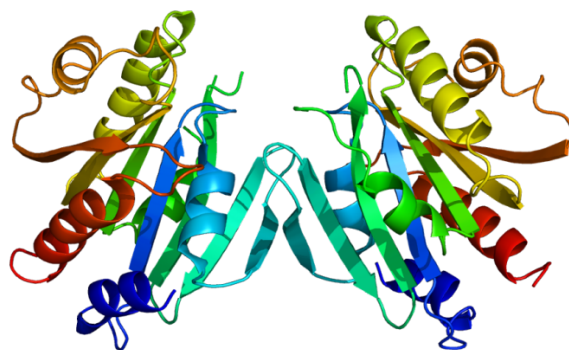




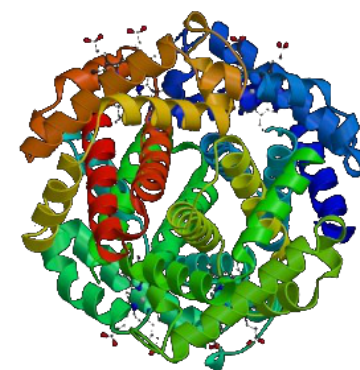




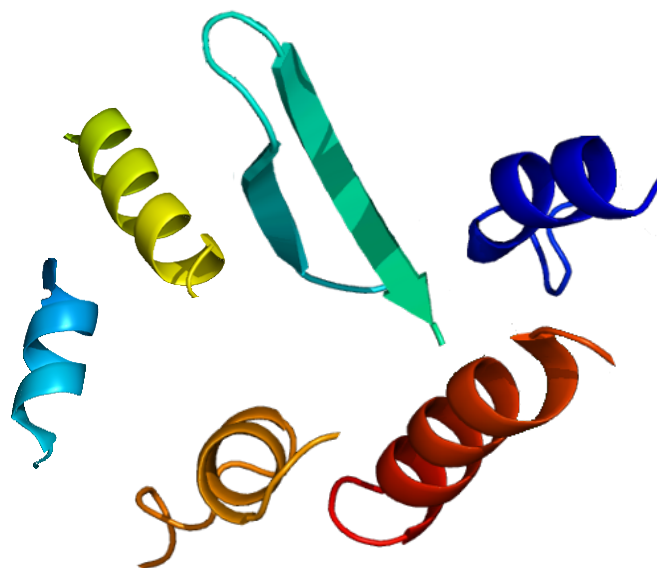
17%



95%



92%





Overview

- Backend with proven potential
- Complete MS analysis implementation
- Abstract to a DSL
- Improved performance





Nesvizhskii10:

“A survey of computational methods and error rate estimation procedures for peptide and protein identification in shotgun proteomics.” Alexey Nesvizhskii, *Journal Of Proteomics*, 2010